

Diseño e Implementación de una CLI Moderna con Comandos "Slash" para Agentes de IA Locales

Informe Técnico de Arquitectura y Viabilidad

Fecha: 28 de Octubre de 2025 **Autor:** Investigador Técnico Senior, Arquitecto de CLIs y Analista UX de TUIs **Versión:** 1.0

1. Resumen Ejecutivo

Este informe presenta un análisis exhaustivo y una arquitectura de referencia para el desarrollo de una interfaz de línea de comandos (CLI) moderna, diseñada para interactuar con agentes de inteligencia artificial locales. La investigación se centra en replicar y mejorar la experiencia de usuario (UX) popularizada por herramientas como Claude Code y Gemini CLI, caracterizada por el uso de "comandos slash" (ej. `/review`). El objetivo es crear una herramienta multi-plataforma y multi-lenguaje, con soporte inicial para proyectos en Python, Java y TypeScript, operando en un entorno completamente local sin dependencias de servicios en la nube.

Hallazgos Clave:

- Patrón Dual de Comandos "Slash":** El estado del arte revela un patrón convergente pero con dos enfoques distintos. Herramientas como Claude Code utilizan archivos Markdown (.md) para definir prompts de formato libre, priorizando la legibilidad humana. Por otro lado, Gemini CLI emplea archivos TOML (.toml) que ofrecen mayor estructura, permitiendo la definición explícita de argumentos y la integración con herramientas externas. Se recomienda una síntesis de ambos, utilizando TOML para la estructura y metadatos, y permitiendo plantillas de prompt embebidas.
- Comunicación Inter-Procesos (IPC) Local:** Para garantizar una experiencia de usuario robusta y sin fricciones en un entorno local, la comunicación entre la CLI y el agente de IA debe evitar la contención de recursos como los puertos de red. El análisis de IPC (HTTP local, gRPC, Unix Sockets, stdio) concluye que un protocolo basado en **JSON-RPC 2.0 sobre stdio** es la opción superior. Este modelo, probado por el Language Server Protocol (LSP), elimina conflictos de puertos, es inherentemente más seguro al no exponer endpoints de red y simplifica la gestión de procesos.
- Arquitectura de Plugins Multi-lenguaje:** La compatibilidad con diversos lenguajes de programación se logra de manera más eficiente a través de una arquitectura de plugins desacoplada. El núcleo de la CLI, escrito en un lenguaje compilado para un rendimiento óptimo (como Go o Rust), actúa como un orquestador que invoca plugins específicos del lenguaje (Node.js, Python, Java) como subprocesos, comunicándose a través del contrato stdio JSON-RPC.

Decisión Recomendada:

Se recomienda proceder con la implementación del proyecto, adoptando la arquitectura de

referencia propuesta. El enfoque inicial debe centrarse en la construcción de un núcleo de CLI ligero y la definición estricta de los contratos de comunicación (IPC y formato de comandos "slash"). Los prototipos de Mínimo Producto Viable (MVP) para TypeScript y Python validarán la arquitectura y sentarán las bases para una expansión futura. La prioridad debe ser la experiencia del desarrollador (DX), la robustez y la seguridad en un entorno local.

2. Arquitectura de Referencia para la CLI de Agente Local

La arquitectura propuesta está diseñada para ser modular, extensible y segura, priorizando un funcionamiento local sin fisuras.

2.1. Diagrama Arquitectónico

El siguiente diagrama ilustra los componentes principales y sus flujos de interacción.

```
graph TD
    subgraph Usuario
        A
    end

    subgraph "CLI (Proceso Principal)"
        A -- "/review src/main.py" --> B[CLI Core]
        B -- "Parsea comando" --> C[Slash Command Engine]
        C -- "Carga '.agent/commands/review.toml'" --> D
        C -- "Construye prompt" --> E[IPC Client (stdio)]
        B -- "Gestiona subprocesso" --> F[Plugin Host]
    end

    subgraph "Plugin (Subproceso por Lenguaje)"
        F -- "Lanza 'python plugin.py'" --> G[Language Plugin (Python)]
        G -- "Recibe petición JSON-RPC" --> H
        E <--> H
        G -- "Lee contexto (archivos, git)" --> I
    end

    subgraph "Agente IA (Proceso Persistente)"
        J
        G -- "Invoca vía IPC" --> J
    end

    style CLI_Core fill:#dae8fc,stroke:#6c8ebf,stroke-width:2px
    style J fill:#d5e8d4,stroke:#82b366,stroke-width:2px
```

Flujo de Ejecución de un Comando:

1. **Entrada del Usuario:** El usuario introduce un comando slash en la terminal (ej. `/review src/main.py`).
2. **Parsing Inicial:** El CLI Core recibe la entrada. El Slash Command Engine identifica que

es un comando slash.

3. **Descubrimiento y Carga:** El motor busca y carga la definición del comando desde el sistema de archivos (ej. `.agent/commands/review.toml`).
4. **Invocación del Plugin:** El Plugin Host determina el lenguaje del proyecto actual (Python) y lanza el Language Plugin correspondiente como un subproceso.
5. **Comunicación IPC:** El IPC Client en el núcleo de la CLI establece una comunicación JSON-RPC sobre los flujos stdio con el IPC Server dentro del plugin. Envía una petición con el prompt construido a partir de la plantilla del comando y los argumentos.
6. **Contextualización:** El Language Plugin recibe la petición, lee el contexto necesario del workspace (el contenido de `src/main.py`) y lo inyecta en el prompt.
7. **Invocación del Agente:** El plugin se comunica con el Local AI Agent (que es un proceso separado y persistente) a través de un mecanismo IPC similar, enviándole el prompt final.
8. **Respuesta:** La respuesta del agente fluye de vuelta a través del plugin hasta la CLI, que la formatea y la presenta al usuario en la terminal.

2.2. Análisis y Selección del Mecanismo de Comunicación Inter-Procesos (IPC)

La comunicación entre la CLI (cliente efímero) y el agente de IA (servicio persistente) es un punto crítico de la arquitectura. La elección del mecanismo IPC impacta directamente en el rendimiento, la robustez, la seguridad y la facilidad de depuración.

Se evaluaron cuatro mecanismos principales para un entorno estrictamente local:

1. **HTTP Local (TCP/localhost):**
 - **Ventajas:** Universalmente compatible y fácil de depurar con herramientas estándar como curl.
 - **Desventajas:** Introduce una sobrecarga de red significativa, incluso en localhost. Es susceptible a conflictos de puertos (EADDRINUSE), una fuente común de frustración para el usuario. Presenta un vector de ataque potencial a través de DNS rebinding si no se implementan cabeceras de seguridad adecuadas `[span_10](start_span)[span_10](end_span)`. Los benchmarks confirman que es la opción más lenta para IPC de alto rendimiento `[span_13](start_span)[span_13](end_span)`.
2. **Unix Domain Sockets (UDS):**
 - **Ventajas:** Ofrece un rendimiento sustancialmente mayor que TCP/localhost al operar directamente a través del kernel del sistema operativo, evitando la pila de red `[span_14](start_span)[span_14](end_span)[span_15](start_span)[span_15](end_span)`. Es un mecanismo seguro y eficiente.
 - **Desventajas:** Su implementación depende del sistema de archivos, lo que requiere gestionar la creación, permisos y limpieza del archivo del socket, lo cual puede ser frágil. No es portable a Windows de forma nativa (aunque existe soporte emulado).
3. **gRPC:**
 - **Ventajas:** Diseñado para alto rendimiento, utilizando HTTP/2 y serialización binaria con Protobuf `[span_16](start_span)[span_16](end_span)`. Proporciona contratos de API fuertemente tipados, generación de código cliente/servidor y soporte nativo para streaming, lo que es ideal para interacciones complejas.
 - **Desventajas:** Introduce una sobrecarga de desarrollo significativa: requiere un

paso de compilación para los archivos .proto y un ecosistema de herramientas más complejo. Para una comunicación puramente local, esta complejidad puede ser excesiva [span_18](start_span)[span_18](end_span).

4. **JSON-RPC 2.0 sobre stdio:**

- **Ventajas:** Este patrón, popularizado por el Language Server Protocol (LSP) [span_19](start_span)[span_19](end_span) y el Model Context Protocol (MCP) [span_11](start_span)[span_11](end_span), elimina por completo la necesidad de gestionar puertos de red o archivos de socket. La CLI lanza el proceso del agente (o plugin) y se comunica directamente a través de sus flujos de entrada y salida estándar. Es inherentemente seguro para un entorno local, ya que no expone ningún endpoint. La gestión del ciclo de vida del proceso hijo es sencilla y robusta.
- **Desventajas:** La depuración puede ser menos directa que con HTTP, ya que requiere interceptar los flujos stdio. Sin embargo, esto se mitiga fácilmente registrando todos los mensajes JSON-RPC en un archivo de trazas.

Decisión Recomendada: Se adopta **JSON-RPC 2.0 sobre stdio** como el mecanismo de comunicación primario. Su simplicidad, robustez y seguridad inherente en un contexto local superan el rendimiento marginalmente superior de UDS o la complejidad de gRPC. Se mantendrá un servidor HTTP local como una opción secundaria, activable mediante un flag, para facilitar la depuración y el desarrollo de agentes por parte de terceros.

2.3. Estrategia Multi-lenguaje y Sistema de Plugins

Para cumplir con el requisito de soportar proyectos en Python, Java y TypeScript, es fundamental una arquitectura de plugins que no acople el núcleo de la CLI a las especificidades de cada lenguaje.

El diseño se basa en el principio de "contrato sobre concreción". El éxito no depende de un formato binario compartido, sino de un contrato de comunicación estricto y agnóstico al lenguaje.

1. **Orquestación por el CLI Core:** El núcleo de la CLI actúa como un orquestador. Al iniciarse en un directorio de proyecto, detecta el tipo de proyecto (buscando package.json, pom.xml, pyproject.toml, etc.) e invoca al Plugin Host.
2. **Lanzamiento de Subprocesos:** El Plugin Host es responsable de encontrar y lanzar el Language Plugin adecuado como un subprocesso (ej. node plugin.js, python plugin.py). Esto aísla las dependencias y el entorno de ejecución del plugin del núcleo de la CLI.
3. **Comunicación Basada en Contrato:** Toda la comunicación entre el CLI Core y el Language Plugin se realiza a través de stdio utilizando el contrato JSON-RPC definido en la sección 5.
4. **Wrappers Ligeros:** Cada Language Plugin es un "wrapper" delgado. Su función es traducir las peticiones genéricas del CLI Core (ej. "leer archivo X", "ejecutar pruebas") en acciones específicas del ecosistema de ese lenguaje, y comunicarse con el Local AI Agent para obtener las respuestas de la IA.

Este modelo, inspirado en arquitecturas como LSP y RPC

[span_21](start_span)[span_21](end_span)[span_22](start_span)[span_22](end_span), ofrece un aislamiento robusto, permite que los plugins se desarrollen y distribuyan de forma independiente, y garantiza que el CLI Core permanezca agnóstico y estable.

3. Tabla Comparativa de Frameworks para CLI/TUI

La elección de los frameworks adecuados es fundamental para la productividad del desarrollo y la calidad de la experiencia final del usuario. La siguiente tabla evalúa los frameworks más relevantes en el estado del arte (últimos 24 meses) para Node.js/TypeScript, Python, Go y Rust, basándose en criterios clave para este proyecto. La evaluación prioriza la experiencia del desarrollador (DX) y la madurez del ecosistema sobre el rendimiento bruto, ya que las diferencias de microsegundos en el parsing de argumentos son insignificantes en comparación con la latencia de los modelos de IA.

Lenguaje	Framework	Rendimiento	DX (Developer Experience)	Ecosistema / Madurez	Soporte Autocompletado	Capacidades TUI	Licencia
Node/TS	oclif	Bueno	Excelente	Muy Alto	Nativo (Plugin)	Limitado (Integración)	MIT
	Commander.js	Muy Bueno	Excelente	Muy Alto	Terceros	Nulo (Integración)	MIT
	yargs	Muy Bueno	Bueno	Muy Alto	Nativo	Nulo (Integración)	MIT
	Ink	N/A (Renderer)	Excelente (React-like)	Alto	N/A	Excelente (Core)	MIT
	blessed	Bueno	Medio (API imperativa)	Medio	N/A	Bueno	MIT
Python	Typer / Click	Muy Bueno	Excelente (Type Hints)	Muy Alto	Nativo	Nulo (Integración)	BSD-3

Lenguaje	Framework	Rendimiento	DX (Developer Experience)	Ecosistema / Madurez	Soporte Autocompletado	Capacidades TUI	Licencia
	span_40](end_span)[span_41](start_span)[span_41](end_span)				nd_span)		
	Rich / Textual span_46](start_span)[span_46](end_span)[span_47](start_span)[span_47](end_span)	N/A (Renderer)	Excelente	Alto	N/A	Excelente (Core)	MIT
	Prompt Toolkit span_48](start_span)[span_48](end_span)	N/A (Input)	Excelente	Alto	Nativo (Avanzado)	Excelente (Input)	BSD-3
Go	Cobra / Viper span_49](start_span)[span_49](end_span)[span_50](start_span)[span_50](end_span)	Excelente	Excelente	Muy Alto	Nativo span_51](start_span)[span_51](end_span)	Limitado (Integración)	Apache-2.0
	Bubble Tea / Lip Gloss span_53](start_span)[span_53](end_span)[span_54](start_span)[span_54](end_span)	N/A (Renderer)	Excelente (Elm Arch)	Alto	N/A	Excelente (Core)	MIT

Lenguaje	Framework	Rendimiento	DX (Developer Experience)	Ecosistema / Madurez	Soporte Autocompletado	Capacidades TUI	Licencia
Rust	clap	Excelente	Excelente	Muy Alto	Nativo (Generación)	Nulo (Integración)	MIT/Apache-2.0
	ratatui	N/A (Renderer)	Excelente	Alto	N/A	Excelente (Core)	MIT

Análisis y Recomendación Justificada:

- **Para el MVP de TypeScript:**
 - **Recomendación:** oclif + Ink.
 - **Justificación:** oclif proporciona una estructura de proyecto robusta, un sistema de plugins maduro y generación de scaffolding, lo que acelera enormemente el desarrollo inicial [span_24](start_span)[span_24](end_span). Su plugin de autocompletado es una solución de primera clase [span_58](start_span)[span_58](end_span)[span_59](start_span)[span_59](end_span). Para la interfaz de usuario, Ink ofrece un modelo declarativo basado en componentes React, lo que permite construir TUI complejas de manera mantenible y familiar para los desarrolladores web [span_36](start_span)[span_36](end_span). La combinación ofrece lo mejor de ambos mundos: una base de CLI sólida y una capa de presentación potente.
- **Para el MVP de Python:**
 - **Recomendación:** Typer + Rich + Prompt Toolkit.
 - **Justificación:** Typer se construye sobre el probado Click y ofrece una experiencia de desarrollo moderna y eficiente al definir la CLI directamente desde las anotaciones de tipo de Python, reduciendo el código boilerplate [span_43](start_span)[span_43](end_span). Para la TUI, se propone un enfoque dual: Rich para renderizar salidas estáticas enriquecidas (tablas, markdown, resaltado de sintaxis) con una API sencilla y potente [span_60](start_span)[span_60](end_span), y Prompt Toolkit para gestionar la entrada interactiva del usuario. Prompt Toolkit es la biblioteca de referencia para crear bucles de prompt avanzados con historial persistente, autocompletado dinámico (de comandos y rutas de archivo) y soporte para key bindings de Emacs/Vi [span_63](start_span)[span_63](end_span)[span_66](start_span)[span_66](end_span). Textual [span_69](start_span)[span_69](end_span) es una alternativa excelente para aplicaciones TUI de pantalla completa, pero este enfoque modular es más flexible para una CLI conversacional.

4. Especificación de Comandos "Slash"

El "lenguaje" de comandos slash es la interfaz principal entre el usuario y el agente. El diseño debe ser intuitivo, extensible y potente. Se propone un formato que sintetiza los patrones observados en Claude Code `` y Gemini CLI [span_1](start_span)[span_1](end_span), adoptando la estructura de TOML por su claridad y la expresividad de las plantillas de prompt.

4.1. Formato de Definición y Descubrimiento

- **Ubicación:** Los comandos se descubrirán en dos ubicaciones, con prioridad para los de proyecto:
 1. **Proyecto:** `.agent/commands/*.toml` (para comandos específicos del proyecto, versionables con Git).
 2. **Usuario:** `~/config/agent/commands/*.toml` (para comandos personales y globales).
- **Formato de Archivo:** Se utilizarán archivos TOML. Cada archivo define un único comando, y su nombre de archivo (ej. `review.toml`) determina el nombre del comando (`/review`). Las subcarpetas crean espacios de nombres (ej. `git/commit.toml` se convierte en `/git:commit`) [span_2](start_span)[span_2](end_span).

Ejemplo de `.agent/commands/review.toml`:

```
# Metadatos para ayuda y parsing
name = "review"
description = "Solicita una revisión de código para un archivo o
número de Pull Request."

# Definición de argumentos para validación y ayuda contextual
args =

# Flags opcionales que modifican el comportamiento
flags =

# Plantilla del prompt que se enviará al agente
[prompt]
template = """
Eres un ingeniero de software senior experto en revisiones de código.
Analiza el siguiente contenido en busca de bugs, vulnerabilidades de
seguridad, problemas de rendimiento y mejoras de estilo y legibilidad.
Proporciona tus comentarios en formato Markdown, utilizando bloques de
código sugeridos para las correcciones.

Contexto del archivo: {{target_path}}
Contenido a revisar:
```{{target_language}}
{{target_content}}
"""
```

## ### 4.2. Sintaxis de Plantillas y Variables

Se utilizará la sintaxis de plantillas ``{{variable}}``, popularizada por herramientas como Gemini CLI ``[span_3](start_span)[span_3](end_span)``. La CLI será responsable de



recolectar y sustituir estas variables antes de invocar al agente.

```
* **Variables de Argumentos:**
* `{{args}}`: Todos los argumentos pasados al comando como una
única cadena.
* `{{arg1}}`, `{{arg2}}`,...: Argumentos posicionales
individuales.
* `{{target}}`: Alias para `{{arg1}}`.

* **Variables de Contexto (inyectadas por la CLI):**
* `{{target_path}}`: La ruta del archivo principal proporcionado
como argumento.
* `{{target_content}}`: El contenido del archivo en
`target_path`.
* `{{target_language}}`: El lenguaje de programación detectado
para `target_path`.
* `{{clipboard_content}}`: El contenido actual del portapapeles
del sistema.
* `{{git_diff}}`: La salida de `git diff` en el repositorio
actual.
```

### ### 4.3. Convenciones de Comandos Base

Se definirá un conjunto inicial de comandos esenciales para proporcionar una funcionalidad básica y consistente:

```
* `/help [command]`: Muestra la ayuda general o de un comando
específico.
* `/model <model_name>`: Selecciona o cambia el modelo de IA activo
para la sesión.
* `/clear`: Limpia el historial de la conversación actual.
* `/plan <task_description>`: Pide al agente que genere un plan de
implementación detallado para una tarea, sin escribir código
`[span_4] (start_span) [span_4] (end_span)`.
* `/review <path|PR#>`: Inicia una revisión de código del objetivo
especificado.
* `/run-tests [test_name]`: Ejecuta los tests del proyecto,
opcionalmente filtrando por nombre.
* `/mem <slice>`: Interactúa con la "memoria" del agente (un
concepto a definir, similar a `CLAUDE.md`
`[span_71] (start_span) [span_71] (end_span)`).
```

### ## 5. Contrato de Interfaz para el Agente de IA Local

Para garantizar el desacoplamiento entre la CLI y las diversas implementaciones de agentes de IA locales, se define un contrato de comunicación estricto, versionado y basado en estándares abiertos.

```
* **Protocolo:** JSON-RPC 2.0 sobre `stdio`
`[span_7] (start_span) [span_7] (end_span) [span_9] (start_span) [span_9] (end_span)`.
* **Codificación:** UTF-8.
```

### ### 5.1. Métodos RPC

```
* **`agent/invoke` (Request):** El método principal para ejecutar una tarea en el agente.
```

```
 * **`params`:**
        ```json  
        {  
            "messages": [ { "role": "user", "content": "..." } ],  
            "tools": ,  
            "constraints": { "max_tokens": 4096 }  
        }  
        ```  
 * **`result`:**
        ```json  
        {  
            "id": "...",  
            "role": "assistant",  
            "content": "...",  
            "tool_calls":  
        }  
        ```
```

```
* **`agent/streamChunk` (Notification):** Notificación enviada desde el agente a la CLI para transmitir partes de una respuesta larga.
```

```
 * **`params`:** `{ "requestId": "...", "chunk": {
"content_delta": "..." } }`
```

```
* **`agent/cancel` (Notification):** Notificación enviada desde la CLI al agente para cancelar una solicitud en curso.
```

```
 * **`params`:** `{ "requestId": "..." }`
```

### ### 5.2. Esquemas de Datos (JSON Schema)

A continuación se definen los esquemas para los objetos de datos principales.

```
`Message`:
```json  
{  
    "type": "object",  
    "properties": {  
        "role": { "enum": ["user", "assistant", "system"] },  
        "content": { "type": "string" }  
    },  
    "required": ["role", "content"]  
}
```

```
}
```

Tool (Definición de una herramienta que la CLI puede ejecutar):

```
{
  "type": "object",
  "properties": {
    "name": { "type": "string" },
    "description": { "type": "string" },
    "parameters": { "$ref": "http://json-schema.org/draft-07/schema#" }
  },
  "required": ["name", "description", "parameters"]
}
```

ToolCall (Solicitud del agente para ejecutar una herramienta):

```
{
  "type": "object",
  "properties": {
    "id": { "type": "string" },
    "toolName": { "type": "string" },
    "args": { "type": "object" }
  },
  "required": ["id", "toolName", "args"]
}
```

Patch (Propuesta de modificación de un archivo):

```
{
  "type": "object",
  "properties": {
    "file_path": { "type": "string" },
    "content_diff": { "type": "string", "description": "Diferencial en formato unificado (diff -u)" }
  },
  "required": ["file_path", "content_diff"]
}
```

Este contrato formaliza la comunicación, permitiendo que cualquier agente que lo implemente pueda integrarse con la CLI, y viceversa.

6. MVP – Prototipo en TypeScript

El MVP en TypeScript servirá para validar la arquitectura central, el sistema de comandos slash y la comunicación IPC.

- **Frameworks:** oclif [span_25](start_span)[span_25](end_span) para la estructura de la CLI y el sistema de plugins. commander.js [span_30](start_span)[span_30](end_span) para el parsing de argumentos (utilizado internamente por oclif). Ink [span_37](start_span)[span_37](end_span) para renderizar componentes de UI como

spinners y salidas formateadas.

6.1. Estructura del Proyecto

```
/
├── src/
│   ├── commands/
│   │   ├── base-command.ts    // Clase base con lógica compartida
│   │   └── slash.ts           // Comando principal que maneja todos
los /...
│   ├── core/
│   │   ├── slash-parser.ts    // Lógica para parsear "/cmd <args>"
│   │   ├── command-loader.ts // Carga y cachea comandos desde.agent/
│   │   └── agent-client.ts     // Cliente JSON-RPC sobre stdio
│   └── hooks/
│       └── init.ts             // Hook de oclif para cargar comandos
al inicio
├── .agent/
│   ├── commands/
│   │   ├── review.toml
│   │   └── optimize.toml
└── package.json
```

6.2. Código Mínimo de Ejemplo

src/commands/slash.ts (Manejador principal de comandos slash):

```
import { Command, Flags } from '@oclif/core';
import { loadCommand } from '../core/command-loader';
import { invokeAgent } from '../core/agent-client';
import { render } from 'ink';
import React from 'react';
import { Spinner } from '@inkjs/ui'; //
[span_72](start_span)[span_72](end_span)

export default class Slash extends Command {
  static description = 'Ejecuta un comando slash para interactuar con
el agente de IA.';
  static strict = false; // Permite argumentos arbitrarios

  public async run(): Promise<void> {
    const { argv } = await this.parse(Slash);
    const fullCommand = argv.join(' ');

    if (!fullCommand.startsWith('/')) {
      this.log('Error: La entrada debe ser un comando slash (ej.
/review <file>).');
      return;
    }
  }
}
```

```

    }

    const commandName = fullCommand.split(' ').substring(1);
    const commandDef = await loadCommand(commandName);

    if (!commandDef) {
        this.log(`Error: Comando desconocido "${commandName}".`);
        return;
    }

    // Lógica para leer archivos, inyectar contexto en la plantilla,
    etc.
    //... (ej. leer el archivo para /review)
    const prompt = `Revisa este código:...`; // Placeholder

    render(<Spinner label="El agente está pensando..." />);

    const response = await invokeAgent({
        messages: [{ role: 'user', content: prompt }],
    });

    // Limpiar el spinner y mostrar la respuesta
    console.log(response.content);
}
}

```

src/core/agent-client.ts (Cliente IPC):

```

import { spawn } from 'child_process';
import { JSONRPCClient } from 'json-rpc-2.0';

// Asumimos que el agente se ejecuta como un subprocesso
const agentProcess = spawn('path/to/agent/executable',, { stdio:
['pipe', 'pipe', 'inherit'] });

const client = new JSONRPCClient((jsonRPCRequest) => {
    agentProcess.stdin.write(JSON.stringify(jsonRPCRequest) + '\n');
});

agentProcess.stdout.on('data', (data) => {
    client.receive(JSON.parse(data.toString()));
});

export async function invokeAgent(params: any): Promise<any> {
    return client.request('agent/invoke', params);
}

```

6.3. Configuración de Autocompletado

Para habilitar el autocompletado de shell, se utilizará el plugin oficial de oclif.

1. Instalar el plugin:

```
npm install @oclif/plugin-autocomplete
```

2. Añadirlo a package.json:

```
{
  "name": "my-cli",
  "version": "0.0.1",
  "oclif": {
    "plugins": [
      "@oclif/plugin-autocomplete"
    ]
  }
}
```

3. Instrucciones para el usuario: El usuario final deberá ejecutar los siguientes comandos para activar el completado en su shell

```
[span_73](start_span)[span_73](end_span)[span_74](start_span)[span_74](end_span):
```

```
# Generar el script de autocompletado
```

```
$ my-cli autocomplete
```

```
# Añadir el script de inicialización a su perfil de shell (ej.
para zsh)
```

```
$ echo "source $(my-cli autocomplete)" >> ~/.zshrc
```

7. MVP – Prototipo en Python

El MVP en Python se centrará en demostrar una experiencia de usuario superior en la terminal, aprovechando bibliotecas especializadas para la entrada y salida.

- **Frameworks:** Typer [span_44](start_span)[span_44](end_span) para una definición de CLI declarativa y moderna. Rich [span_61](start_span)[span_61](end_span) para renderizar salidas con formato (Markdown, tablas, resaltado de sintaxis). Prompt Toolkit [span_64](start_span)[span_64](end_span)[span_67](start_span)[span_67](end_span) para crear un bucle de prompt interactivo y avanzado.

7.1. Estructura del Proyecto

```
/
├── my_cli/
│   ├── __main__.py          // Punto de entrada principal con el bucle
de prompt
│   ├── core/
│   │   ├── command_loader.py
│   │   └── agent_client.py
```

```

|   └─ commands.py           // Lógica para manejar comandos como
/review
└─ .agent/
   └─ commands/
      └─ review.toml
      └─ optimize.toml
└─ pyproject.toml

```

7.2. Código Mínimo de Ejemplo

my_cli/__main__.py (Bucle de prompt interactivo):

```

import typer
from prompt_toolkit import PromptSession
from prompt_toolkit.history import FileHistory
from prompt_toolkit.auto_suggest import AutoSuggestFromHistory
from prompt_toolkit.completion import WordCompleter
from rich.console import Console
from .commands import handle_command

# Lista de comandos cargados desde .agent/commands/
COMMANDS = ["/review", "/optimize", "/help", "/clear"]

def main():
    session = PromptSession(history=FileHistory("~/.agent_history"))
    console = Console()
    completer = WordCompleter(COMMANDS, ignore_case=True)

    while True:
        try:
            user_input = session.prompt(
                "> ",
                auto_suggest=AutoSuggestFromHistory(),
                completer=completer,
                complete_while_typing=True,
            )
            if user_input.strip():
                handle_command(user_input, console)
        except (KeyboardInterrupt, EOFError):
            break

if __name__ == "__main__":
    typer.run(main)

```

my_cli/commands.py (Lógica de comando y renderizado con Rich):

```

from rich.console import Console
from rich.markdown import Markdown
from rich.progress import Progress

```

```

from core.agent_client import invoke_agent
from core.command_loader import load_command_template

def handle_command(user_input: str, console: Console):
    parts = user_input.strip().split()
    command_name = parts

    if command_name == "/review":
        if len(parts) < 2:
            console.print("[bold red]Error: El comando /review
requiere una ruta de archivo.[/bold red]")
            return

        file_path = parts
        try:
            with open(file_path, "r") as f:
                content = f.read()
        except FileNotFoundError:
            console.print(f"[bold red]Error: Archivo no encontrado en
'{file_path}'.[/bold red]")
            return

        template = load_command_template("review")
        prompt = template.replace("{{target_content}}", content)

        with Progress(console=console) as progress:
            task = progress.add_task("[cyan]Invocando al agente...",
total=None)
            response = invoke_agent({"messages": [{"role": "user",
"content": prompt}]}))
            progress.stop()

        # Renderizar la respuesta del agente como Markdown
        md = Markdown(response["content"])
        console.print(md)
    else:
        console.print(f"[yellow]Comando '{command_name}' no
reconocido.[/yellow]")

```

8. Seguridad y Barreras de Protección (Guardrails)

Dado que el agente de IA puede proponer acciones que modifican el sistema de archivos o ejecutan comandos de shell, la CLI debe operar bajo un modelo de **confianza cero**. La seguridad no puede ser delegada al agente; la CLI debe actuar como un intermediario de confianza que impone políticas estrictas antes de ejecutar cualquier acción potencialmente peligrosa [span_75](start_span)[span_75](end_span).

- **Confirmación Interactiva Obligatoria:** Cualquier acción que modifique el estado del sistema fuera del propio historial de la CLI requerirá una confirmación explícita del usuario. Esto incluye:
 - Escribir o modificar archivos (Patch).
 - Ejecutar comandos de shell que no estén en una lista blanca (ej. git commit, npm install, docker run).
 - La solicitud de confirmación mostrará claramente la acción a realizar y el usuario deberá responder con y o yes.
- **Modo de Simulación (--dry-run):** Un flag global --dry-run permitirá a los usuarios ver qué acciones habría realizado el agente sin ejecutarlas realmente. La salida indicará claramente `` antes de cada acción simulada.
- **Lista Blanca de Comandos (Allow-list):** Se mantendrá un archivo de configuración (~/.config/agent/settings.toml) donde los usuarios pueden definir una lista de comandos de shell considerados seguros para ejecutarse sin confirmación (ej. ls -la, cat, grep). Por defecto, esta lista estará vacía.
- **Aislamiento del Directorio de Trabajo (CWD Jail):** Todas las operaciones de archivos y la ejecución de comandos estarán, por defecto, restringidas al directorio de trabajo actual del proyecto y sus subdirectorios. El acceso a rutas fuera de este "jail" (ej. /etc/, ~/) requerirá un flag explícito (ej. --allow-root-access) y una confirmación adicional.
- **"No-Mess-Left-Behind":** La CLI implementará una gestión robusta de archivos temporales y mecanismos de try-catch-finally para garantizar que los recursos se limpien adecuadamente, incluso si una operación falla a mitad de camino.

9. Estrategias de Pruebas y Observabilidad

Para garantizar la fiabilidad y mantenibilidad de la CLI, se implementarán estrategias robustas de pruebas y observabilidad local.

- **Pruebas End-to-End (E2E) con "Golden Prompts":** El enfoque principal para las pruebas E2E se basará en el concepto de "Golden Prompts" [span_78](start_span)[span_78](end_span)[span_79](start_span)[span_79](end_span). Se creará un conjunto de datos de prueba que consiste en:
 1. Un prompt de entrada canónico (ej. una solicitud de revisión para un fragmento de código específico).
 2. La salida esperada del agente en un formato estable (el "golden file"). Las pruebas automatizadas ejecutarán estos prompts contra una versión simulada (fake) del agente de IA que devuelve respuestas predefinidas. El resultado se comparará con el "golden file" correspondiente para detectar cualquier regresión en el parsing, el formateo de la salida o la lógica de la CLI.
- **Observabilidad y Trazabilidad Local:** Para facilitar la depuración y el análisis del comportamiento de la herramienta, toda la actividad se registrará localmente de forma estructurada.
 1. **Logging Estructurado (JSONL):** Todas las interacciones significativas (inicio de sesión, ejecución de comando, petición a IPC, respuesta recibida) se registrarán en un archivo local (~/.cache/agent/logs.jsonl) en formato JSON por línea (JSONL) [span_80](start_span)[span_80](end_span)[span_81](start_span)[span_81](end_span)[span_82](start_span)[span_82](end_span). Este formato es fácil de parsear y consultar con herramientas como jq o klp

- [span_83](start_span)[span_83](end_span).
2. **IDs de Correlación:** Cada sesión de la CLI generará un session_id único. Cada comando slash ejecutado generará un request_id. Estos IDs se incluirán en cada línea de log y se propagarán a través de todas las llamadas IPC, permitiendo trazar una interacción completa desde la entrada del usuario hasta la respuesta final del agente.
 3. **Métricas Clave:** Cada registro de log asociado a una invocación del agente incluirá métricas esenciales: latencia total de la respuesta, número de tokens de entrada y salida (si el agente los proporciona), y el estado de éxito o error de la operación.

10. Hoja de Ruta de Implementación (30-60-90 Días)

Se propone una hoja de ruta agresiva pero realista para entregar valor de forma incremental.

- **Días 1-30: Fundación y MVP Core (TypeScript)**
 - **Objetivos:** Solidificar la arquitectura, el contrato IPC y el formato de comandos .toml. Desarrollar el MVP funcional en TypeScript.
 - **Entregables Clave:**
 - Documento de arquitectura y diseño finalizado y aprobado.
 - Esquemas JSON para el contrato IPC v1.0.
 - CLI en TypeScript instalable vía npm, con los comandos /help, /review <file> y /optimize <file>.
 - Comunicación funcional con un agente simulado (mock) que responde a través de stdio.
 - Repositorio Git con CI básica (linting, tests unitarios).
 - **Días 31-60: Expansión Multi-lenguaje y Seguridad**
 - **Objetivos:** Validar la arquitectura de plugins con el MVP de Python e implementar las barreras de seguridad fundamentales.
 - **Entregables Clave:**
 - CLI en Python instalable vía pipx, con la UI enriquecida (Rich + Prompt Toolkit).
 - Implementación del Plugin Host en el CLI Core capaz de detectar y lanzar los plugins de TS y Python.
 - Implementación completa del modelo de seguridad: confirmaciones interactivas, modo --dry-run y CWD jail.
 - Inicio de la integración con un agente de IA local real (ej. MemTech).
 - **Días 61-90: Refinamiento de UX, Distribución y Comandos Avanzados**
 - **Objetivos:** Mejorar la experiencia de usuario, establecer canales de distribución robustos y ampliar la funcionalidad.
 - **Entregables Clave:**
 - TUI mejorada con una paleta de comandos (fuzzy finder) para descubrir y ejecutar comandos slash.
 - Implementación de los comandos /plan y /run-tests.
 - Pipelines de CI/CD para publicar automáticamente paquetes en npm y PyPI (para pipx).
 - Creación y publicación de una fórmula de Homebrew para una instalación sencilla en macOS y Linux
- [span_84](start_span)[span_84](end_span)[span_85](start_span)[span_85](e

- nd_span).
- Documentación inicial para usuarios y desarrolladores de plugins.

11. Riesgos y Mitigaciones

A continuación se presenta una matriz que identifica los riesgos principales del proyecto, junto con su probabilidad, impacto y las estrategias de mitigación propuestas.

Riesgo	Probabilidad	Impacto	Estrategia de Mitigación
Fragmentación del Ecosistema de Agentes Locales	Alta	Medio	Diseñar un contrato IPC simple, bien documentado y basado en estándares (JSON-RPC) para facilitar la creación de adaptadores para diferentes agentes. Proporcionar un agente de referencia o simulado para el desarrollo.
Bajo Rendimiento del Agente Local en Hardware Modesto	Media	Alto	Implementar timeouts configurables y agresivos en el cliente IPC. Permitir al usuario configurar límites de recursos para los modelos. Promover y documentar el uso de modelos cuantizados y optimizados para CPU.
Complejidad en la Gestión de Dependencias Multi-lenguaje	Media	Medio	Utilizar instaladores de aplicaciones como pipx y npm i -g para aislar las dependencias de la CLI de los proyectos del usuario [span_86](start_span)[span_86](end_span). Documentar claramente los runtimes requeridos (Node.js, Python, JVM).
Vulnerabilidades de Seguridad por Ejecución de Código	Baja	Crítico	Implementar un modelo de seguridad estricto de "confianza cero" con confirmaciones

Riesgo	Probabilidad	Impacto	Estrategia de Mitigación
			interactivas por defecto, listas blancas configurables y sandboxing del directorio de trabajo, como se detalla en la sección 8 [span_76](start_span)[span_76](end_span).
Adopción Lenta por Parte de los Desarrolladores	Media	Alto	Priorizar la experiencia del desarrollador (DX) desde el día uno: instalación sencilla (Homebrew), autocompletado robusto, documentación clara y comandos intuitivos.

12. Referencias

A continuación se listan las fuentes primarias consultadas para la elaboración de este informe, con sus respectivas fechas de acceso.

1. Anthropic. "Slash Commands - Claude Code Documentation". *docs.claude.com*, consultado el 25 de octubre de 2025. ``
2. Google Cloud. "Gemini CLI custom slash commands". *cloud.google.com*, 30 de julio de 2025. [span_5](start_span)[span_5](end_span)
3. Charmbracelet. "Bubble Tea Package Documentation". *pkg.go.dev*, consultado el 25 de octubre de 2025. [span_87](start_span)[span_87](end_span)
4. Vadim Demedes. "Ink - React for CLIs". *GitHub*, consultado el 25 de octubre de 2025. [span_38](start_span)[span_38](end_span)
5. Textualize. "Textual Documentation". *textual.textualize.io*, consultado el 25 de octubre de 2025. [span_70](start_span)[span_70](end_span)
6. Read the Docs. "Rich Documentation". *rich.readthedocs.io*, consultado el 25 de octubre de 2025. [span_62](start_span)[span_62](end_span)
7. Salesforce. "oclif - The Open CLI Framework". *oclif.io*, consultado el 25 de octubre de 2025. [span_26](start_span)[span_26](end_span)
8. Sebastián Ramírez. "Typer Tutorial". *typer.tiangolo.com*, consultado el 25 de octubre de 2025. [span_45](start_span)[span_45](end_span)
9. Jonathan Slenders. "prompt-toolkit Documentation". *python-prompt-toolkit.readthedocs.io*, consultado el 25 de octubre de 2025. [span_65](start_span)[span_65](end_span)[span_68](start_span)[span_68](end_span)
10. ACPEX. "Agent Client Protocol (ACP) Overview". *hexdocs.pm*, consultado el 25 de octubre de 2025. [span_20](start_span)[span_20](end_span)
11. Model Context Protocol. "MCP Specification - Transports". *modelcontextprotocol.io*, 18 de junio de 2025. [span_12](start_span)[span_12](end_span)

12. John-David Dalton. "Commander.js Documentation". *npmjs.com*, consultado el 25 de octubre de 2025. [span_31](start_span)[span_31](end_span)
13. Steve Francia. "Cobra - A Commander for modern Go CLI interactions". *GitHub*, consultado el 25 de octubre de 2025. [span_88](start_span)[span_88](end_span)
14. Steve Francia. "Viper - Go configuration with fangs". *GitHub*, consultado el 25 de octubre de 2025. [span_89](start_span)[span_89](end_span)
15. The clap-rs Authors. "clap Documentation". *prisma.github.io*, consultado el 25 de octubre de 2025. [span_56](start_span)[span_56](end_span)
16. oclif team. "@oclif/plugin-autocomplete". *npmjs.com*, consultado el 26 de octubre de 2025. [span_28](start_span)[span_28](end_span)
17. Cobra Authors. "Shell Completion How-to Guide". *cobra.dev*, consultado el 26 de octubre de 2025. [span_52](start_span)[span_52](end_span)
18. Microsoft. "gRPC vs HTTP APIs". *learn.microsoft.com*, consultado el 26 de octubre de 2025. [span_17](start_span)[span_17](end_span)
19. Arroyo Systems. "Plugin Systems in Rust". *arroyo.dev*, consultado el 26 de octubre de 2025. [span_90](start_span)[span_90](end_span)
20. WorkOS. "Best practices for CLI authentication". *workos.com*, consultado el 27 de octubre de 2025. [span_77](start_span)[span_77](end_span)
21. Better Stack. "Logging in Go with Slog: A Detailed Guide". *betterstack.com*, consultado el 27 de octubre de 2025. [span_91](start_span)[span_91](end_span)
22. New Relic. "Guide to structured logging in Python". *newrelic.com*, 16 de mayo de 2025. [span_92](start_span)[span_92](end_span)